AID-SQL: Adaptive In-Context Learning of Text-to-SQL with Difficulty-Aware Instruction and Retrieval-Augmented Generation

Xiuwen Li*, Qifeng Cai*, Yang Shu✉, Chenjuan Guo, Bin Yang
School of Data Science and Engineering, East China Normal University
Shanghai, China
{xwli, qfcai}@stu.ecnu.edu.cn, {yshu, cjguo, byang}@dase.ecnu.edu.cn

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00294

***Abstract*—Recent research in Text-to-SQL translation has primarily adopted in-context learning methods leveraging large language models (LLMs), achieving significant progress. However, these methods face challenges in adapting to natural language questions of varying difficulty and the relevance of the few-shot examples provided. In this paper, we propose an adaptive in-context learning approach with difficulty-aware instruction and retrieval-augmented generation to enhance the performance of Text-to-SQL translation (AID-SQL). First, we introduce adaptive instructions for LLMs, which employ precise difficulty classification to apply difficulty-adaptive generative guidelines and chain of thought (CoT) templates for varying difficulty levels. We automatically incorporate few-shot examples retrieved through the knowledge base into the CoT template to construct CoT-enhanced examples, which improves the capability of LLMs with retrieval-augmented generation (RAG). Furthermore, considering that current RAG methods struggle to effectively measure the contribution of retrieved examples in solving the specific task of Text-to-SQL translation, we train a ranking model that can better bridge the semantic and structural gap between NL questions and SQL queries. This approach can better understand semantic information and allows for retrieving examples that are more beneficial to the final problem-solving. We evaluate our method on five benchmarks. Our method achieves competitive performance compared with existing methods.**

***Index Terms*—Text-to-SQL, Large Language Model, SQL**

I. Introduction

With the rapid advancement of information technology, databases have become essential tools for storing and managing vast amounts of data [1]. Structured Query Language (SQL) allows users to access and manipulate databases flexibly. However, the complexity of SQL queries presents a significant barrier for non-experts, limiting their ability to leverage database resources effectively. To address this issue, the Text-to-SQL translation has emerged, aiming to automatically convert the natural language question (NL question) into an executable SQL query. This provides a more user-friendly interface for interacting with relational databases, enabling users to perform queries without expertise in SQL.

The research of Text-to-SQL translation has evolved from early rule-based and predefined template methods [2]–[4], to approaches based on deep language models, undergoing multiple iterations and developments. Sequence-to-sequence models [5]–[7] and fine-tuned pre-trained models such as BERT [8], [9] and T5 [10] have demonstrated significant advancement on Text-to-SQL translation.

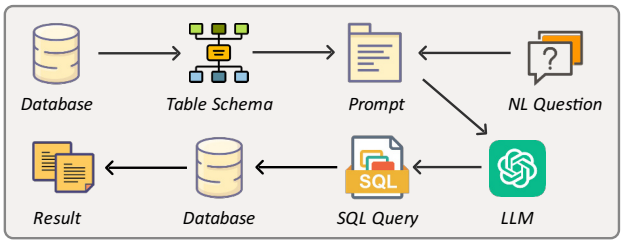


Fig. 1: The basic process of the Text-to-SQL translation

Recent advanced solutions based on LLM have further enhanced the performance of Text-to-SQL translation [11], [12]. As shown in Fig. 1, the prompt, which consists of NL question, the table schema of the corresponding database, and the instruction, is fed into the LLM to generate SQL queries. The final result is obtained by executing the generated SQL query in the database and returning it to the user. Among methods leveraging LLM, leading solutions adopt in-context learning approaches by providing a degree of prior knowledge to guide LLMs in translating NL questions to SQL [13], [14]. This in-context learning approach based on LLMs has emerged as a new paradigm for solving Text-to-SQL translation.

Two aspects greatly influence the effectiveness of in-context learning for Text-to-SQL translation: (1) *Instructions:* Instructions represent input into LLMs, guiding LLMs to analyze and solve Text-to-SQL translation. Numerous techniques are employed, including the design of guiding formats [15], the Chain-of-Thought structure [16], [17], schema linking [18], and so on. Since LLMs exhibit varying performance when handling SQL queries of different difficulties [19], difficulty-adaptive instructions become crucial. However, existing studies often employ fixed and simple instructions, which severely limit the translation performance of LLMs when dealing with target problems of varying difficulty. For complex problems, overly simple prompts cannot adequately analyze and construct the complex structures needed for SQL queries; for simple problems, complex prompts may mislead the translation

*Equal Contribution. ✉ Corresponding Author.

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00294

Authorized licensed use limited to: ULAKBIM UASL - SABANCI UNIVERSITY. Downloaded on October 12,2026 at 15:59:47 UTC from IEEE Xplore. Restrictions apply.

process of LLMs. (2) *Few-shot Examples:* Few-shot examples provide demonstration examples for the translation process. When instructing LLMs to solve Text-to-SQL translation, the retrieval of few-shot examples significantly impacts the results. However, the current few-shot example retrieval strategies fail to accurately capture the extent to which examples facilitate problem-solving, and the metric for retrieval does not align with the ground-truth similarity that would genuinely aid in problem resolution. Traditional studies rely on manually selecting fixed representative examples, which lack adaptability to various problems and scalability. In contrast, some studies employ question-question similarity to retrieve examples, but this approach lacks a multi-dimensional consideration, leading to insufficient representativeness of similarity. Meanwhile, most retrieval methodologies are overly generalized, failing to effectively match the specific similarity required within the Text-to-SQL translation task.

In response to the above problems, we propose an adaptive in-context learning method called AID-SQL, which uses difficulty-aware instruction and retrieval-augmented generation. We also propose a difficulty-aware instruction strategy to fully leverage the capabilities of LLMs in handling translation tasks of different difficulties. Our approach first classifies the given NL problems by difficulty and then utilizes adaptive generative guidelines and chain-of-thought (CoT) templates according to the difficulty category.

To address the issue where few-shot examples fail to genuinely aid SQL generation, we propose a retrieval strategy based on a dedicated ranking model. For the aspect of few-shot example retrieval, we first design an approximate measurement of example quality for different target problems, then use this measurement to train the ranking model. This model compresses text into a type of vector sequence representation, which is a fine-grained type designed to capture the semantic relation between the NL question and SQL. Such improvements ensure we obtain beneficial few-shot examples of higher quality to enhance dynamic CoT prompt construction. After obtaining the few-shot examples through the retrieval strategy, our adaptive method dynamically constructs CoT-enhanced examples, using the CoT template determined by the difficulty level, and finally integrates the NL question, database information, generative guidelines, CoT-enhanced examples, and CoT template as prompt inputs to LLMs. The main contributions of this paper are as follows:

- **We propose difficulty-aware instruction based on the difficulty classification of the target problem.** Our method utilizes a classification model to categorize NL questions based on their difficulty. We then design difficulty-aware generative guidelines and CoT templates tailored for each difficulty level, which can be combined with few-shot examples obtained through our retrieval strategy. This instructional framework proves effective in addressing Text-to-SQL translation.
- **We train a dedicated ranking model to retrieve few-shot examples with higher contributions to the Text-to-SQL translation.** We trained a ranking model that better bridges the semantic and structural gap between NL questions and SQL queries, achieving superior precision among existing example retrieval methods.
- **We evaluate our method through comprehensive experiments.** Experiments are done across five benchmarks. Our method demonstrates competitive performance across these benchmarks, which validates the effectiveness of our approach.

II. Related Works

Research on Text-to-SQL falls into four categories: *rule-driven methods*, *Seq2Seq-based models*, *fine-tuning based on pre-trained language models*, and *methods leveraging LLMs*.

A. Rule-driven Methods

Rule-driven methods are early attempts at Text-to-SQL tasks. These methods generate SQL queries via pre-defined templates and perform well in specific domains [2]–[4]. However, their application is limited due to the complex rules and difficulty in generalizing across domains. Differences in database syntax and domain-specific terminology can make the rules challenging to apply universally.

B. Seq2Seq Model Based Methods

Seq2Seq-based methods [5] adopt encoder-decoder architectures to translate natural language questions into SQL queries. Early approaches, such as Seq2SQL [6], relied on RNN or LSTM-based models but encountered difficulties in handling complex queries and intricate database structures. Recent advancements have introduced more sophisticated techniques, including graph-based methods like ShadowGNN [20], which represent database schemas as graphs to capture their structural properties. Additionally, hybrid frameworks such as IRNet [21] leverage relational embeddings to model relationships between schema elements, while RAT-SQL [22] enhances schema understanding through relational attention.

C. Fine-Tuning of Pre-trained Language Models

The fine-tuning of pre-trained language models, such as BERT [8], [9] and T5 [10], has significantly improved performance in Text-to-SQL tasks. By leveraging pre-training on large-scale corpora and task-specific fine-tuning, these methods effectively capture the complex mapping between natural language questions and database schemas [23], [24]. While fine-tuning approaches demonstrate strong capabilities across diverse tasks, their reliance on high-quality annotated data and susceptibility to overfitting limit their generalization in complex scenarios.

D. Methods Leveraging LLMs

Methods using LLMs have recently demonstrated significant potential in Text-to-SQL translation. Through in-context learning methods, LLMs can achieve remarkable outcomes even in zero-shot settings [25], [26]. Models like ChatGPT and GPT-4, with a well-designed prompt, can generate high-quality SQL queries without extensive labeled data [15]. Advanced

Authorized licensed use limited to: ULAKBIM UASL - SABANCI UNIVERSITY. Downloaded on October 12,2026 at 15:59:47 UTC from IEEE Xplore. Restrictions apply.

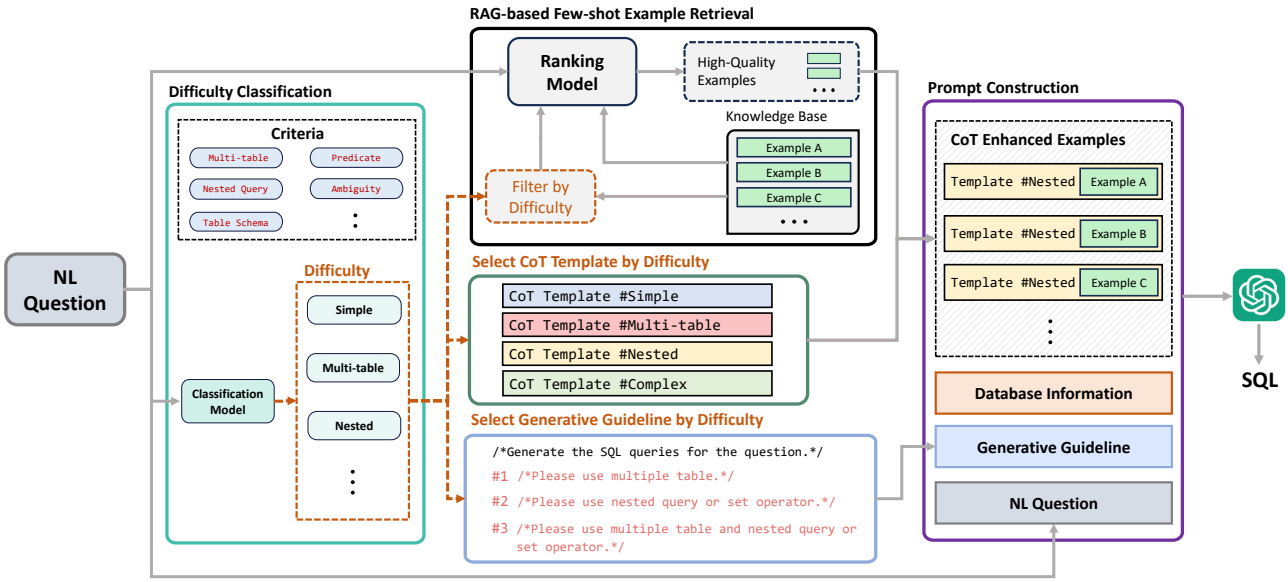


Fig. 2: The overview of our framework.

techniques, such as task decomposition [19], further improve model performance.

Recent advancements have increasingly employed in-context learning methods, where the retrieval of examples significantly impacts the outcomes of LLMs. Improving the similarity of few-shot examples has enhanced the performance of Text-to-SQL tasks. In prior research on Text-to-SQL, two primary approaches have emerged for example retrieval. (1)The first compares the similarity between the target NL question and NL question of examples [26], which lacks accuracy. (2)The second approach pre-generates SQL for the target problem and then compares the similarity between this pre-generated SQL and the SQL queries of the examples [27], which may result in the loss of information pertinent to the question text. DAIL-SQL [15] combines these two methods, thereby enhancing the accuracy of example retrieval. Nevertheless, these text similarity comparison methods primarily employ a Bi-Encoder structure via a pre-trained language model [28], which extracts information from text inputs into a single vector representation. This approach can lead to information loss due to the compression of a vector. Moreover, the complexity of prompt design and the high computational demands of LLMs remain significant challenges for their widespread adoption in practical applications. Numerous approaches have also fine-tuned open-source LLMs to make them more adept at Text-to-SQL translation, thus laying the groundwork for further progress in related research. [15], [29]

III. Methodology

A. Overall Framework

In this paper, we propose an adaptive in-context learning method to enhance the performance of LLMs in Text-to-SQL translation, which is called AID-SQL. As shown in Fig. 2, the framework of our method consists of two main modules: (1) Difficulty-Aware Prompt and (2) Few-shot Example Retrieval for Retrieval-augmented Generation (RAG).

The Difficulty-Aware Prompt Module includes the difficulty classification, CoT template, generative guideline selection, and prompt construction sections depicted in Fig. 2. There exist significant differences among target problems solved. To this end, we designed a framework where SQL queries are classified based on different criteria and designed instructions corresponding to different classifications. Our work considers the basic structure criterion for designing corresponding instructions. This approach guides the LLM to generate SQL queries more accurately and enhances its generality in handling diverse generation tasks. We first utilize a trained difficulty classification model to determine the difficulty category of the NL question, based on which we select the generative guidelines and CoT template. We collect the few-shot examples and dynamically incorporate examples into the CoT template corresponding to the difficulty of the NL question, creating CoT-enhanced examples. Ultimately, the question text, database information, generative guidelines, and the enhanced CoT example together form a complete prompt.

During this process, the examples are obtained through a retrieval process to further enhance the performance of LLMs with retrieval augmented generation, which is shown as the few-shot example retrieval section in Fig. 2. Given a target NL question, this module measures the quality of all candidate examples using our dedicated ranking model, which is trained on a well-designed Text-to-SQL ranking dataset and has a strong ability to capture similarities between examples and the target problems tailored to Text-to-SQL tasks. After getting the measurements of example qualities, the module retrieves a set of the highest-quality ones as few-shot examples.

Authorized licensed use limited to: ULAKBIM UASL - SABANCI UNIVERSITY. Downloaded on October 12,2026 at 15:59:47 UTC from IEEE Xplore. Restrictions apply.

B. Difficulty-Aware Prompt

This section aims to provide difficulty-adaptive prompts for Text-to-SQL translation based on the difficulty classification of the NL questions. It consists of three parts: (1) Question Difficulty Classification, (2) Difficulty-Aware Guidelines, and (3) CoT-Enhanced Example Generation.

Question Difficulty Classification. Given the inherent variations in SQL query difficulty levels, employing differentiated generation strategies becomes essential. We propose a framework where SQL queries are classified based on difficulty criteria and design instructions corresponding to different classifications. However, the difficulty criteria of existing benchmarks fail to provide sufficient guidance for the correct generation of SQL. For example, Spider [12] classifies SQL complexity from a component-based perspective, determining the difficulty level based on the number of components contained in the SQL statement. BIRD [30] further incorporates comprehension difficulty, external knowledge, and schema complexity into its evaluation. However, these designs based on component count or external factors exhibit limitations in practical SQL generation: SQL statements at the same difficulty level may have entirely different structures, leading to vastly different generation logic. For example, an SQL statement containing EXCEPT and another containing nested queries might be classified at the same difficulty level, yet their generation logic is distinct. Such classification criteria fail to provide unified guidance for SQL generation of varying difficulty levels, diminishing their practical value in guiding real-world SQL generation tasks.

Multiple criteria, including the number of predicates, question ambiguity, and database schema characteristics, can influence the generation logic of SQL queries. However, exploring the precise mechanisms of their impact is currently challenging. Here we adopt a basic implementation approach, selecting two fundamental dimensions as our classification criteria. The first dimension is whether multiple tables are involved. Single-table queries exhibit simple structures and straightforward thought processes, whereas multi-table queries require joining multiple tables, significantly increasing their complexity and demanding more sophisticated logical reasoning. Additionally, single-table queries can be illustrated with simple examples, while multi-table queries necessitate more complex examples to help the LLM understand table relationships and join conditions.

The second dimension is whether nested queries are utilized. Nested queries decompose complex problems into simpler components, adopting a local-to-global strategy that solves individual subproblems before integrating them into a comprehensive solution. This dimension complements the single/multi-table classification, enabling a more thorough analysis of complex SQL structures that are particularly challenging for LLMs. Proper guidance on nested queries reduces the difficulty of query formulation and significantly enhances LLMs' accuracy in processing complex queries. These two structural distinctions allow for a reasonable and effective categorization of SQL queries. It should be noted that for more complex scenarios, our framework can be adaptively extended by incorporating additional criteria and customized instruction designs, thereby enhancing the generalization capability of the framework.

Based on our classification criteria, we can design targeted strategies to guide the LLM in generating different types of SQL queries accurately. The difficulty level of SQL queries is determined as follows:

- $d_{\text{multi-table}}$: whether multiple tables are used.
- d_{nested}: whether nested queries are used.

where $d_{\text{multi-table}} \in \{0, 1\}$ and $d_{\text{nested}} \in \{0, 1\}$. These two key dimensions determine the specific type and structure of a SQL statement, based on which the specific classification logic is defined as follows:

- **Simple Category**: These SQL queries do not involve multiple tables, nor do they include nested queries. Typically, such queries are limited to single-table operations, are straightforward in structure, and have a low parsing difficulty.
- **Multi-table Category**: This category comprises queries that involve multiple tables but exclude nested queries. These queries necessitate the integration of several tables, increasing the complexity of the query. For this difficulty, $d_{\text{multi-table}} = 1$ and $d_{\text{nested}} = 0$.
- **Nested Category**: SQL Queries in this category do not involve multiple tables but encompass nested queries. These queries require more advanced logical processing and thus exhibit a greater depth in query structure. Here, $d_{\text{multi-table}} = 0$ and $d_{\text{nested}} = 1$.
- **Complex Category**: This category includes queries involving both multiple tables and nested queries. These queries integrate complex table associations with intricate query logic to represent the pinnacle of structural complexity and parsing difficulty. For this difficulty, $d_{\text{multi-table}} = 1$ and $d_{\text{nested}} = 1$.

We use d to represent the difficulty category of SQL query, where $d \in \{0, 1, 2, 3\}$, corresponds to the four difficulty levels defined above.

When classifying SQL queries, our main challenge is that the only available information is the NL question q itself, and the correct classification relies on analyzing the ground-truth SQL query s_t, which we cannot access. The NL question contains information about the type of SQL query. For example, when an NL question explicitly mentions multiple entities, the corresponding SQL typically involves the operation of multiple tables. When an NL question contains sub-questions or complex conditional logic, it usually involves nested queries. Moreover, due to semantic and structural differences between NL questions and SQL queries, relying solely on the NL question for difficulty classification is insufficient; it also requires information from the SQL query. In the absence of s_t, we use a pre-trained language model (PLM) to pre-generate an SQL query s_p to represent the information contained in s_t.

Authorized licensed use limited to: ULAKBIM UASL - SABANCI UNIVERSITY. Downloaded on October 12,2026 at 15:59:47 UTC from IEEE Xplore. Restrictions apply.

The process is represented as follows:

$$s_p = \mathcal{PGM}(q, D) \tag{1}$$

$\mathcal{PGM}$ denotes the pre-generation model call used for the generation of s_p and D denotes the database information, such as table schema and foreign keys. Using a difficulty classification model DCM, we established a multi-class task to categorize q into four difficulty categories defined above. The DCM is based on a pre-trained language model PLM but not the same one used for pre-generating s_p. The classification process is as follows:

$$\hat{d} = \mathcal{DCM}(q, s_p) \tag{2}$$

Here, $\hat{d}$ is the predicted difficulty of q, and DCM denotes difficulty classification model call. s_p and the question q together serve as inputs for difficulty classification.

The DCM is constructed by adding a classification layer to a PLM and fine-tuning it. The training data comes from the knowledge base B, which is constructed from the benchmarks' training datasets. For each example e_i, $i = 1, 2, \ldots, N$, the data pair $\langle q_i, s_i \rangle$ is used, where q_i denotes the NL question of e_i and s_i denotes the SQL query of e_i. The difficulty d_i corresponding to q_i is obtained through the structural analysis of s_i and can be expressed as $d_i = \mathcal{A}(s_i)$. The function $\mathcal{A}$ here represents the SQL structural analysis process. Through the above process, we construct a training dataset $\{(\langle q_i, s_i \rangle, d_i)\}$. The training loss function L_{cla} is designed as follows:

$$L_{cla} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{4} \mathcal{I}(c = d_i) \log(p_{ic}) \tag{3}$$

N is the total number of examples used for training, $\mathcal{I}$ represents an indicator function, d_i represents the ground-truth difficulty, and p_{ic} represents the probability that the input $\langle q_i, s_i \rangle$ to the difficulty classification model belongs to difficulty category c.

Difficulty-Aware Guidelines. In the process of constructing SQL queries, one challenge that LLMs face is that they lack specific execution directions to complete the SQL generation. This limitation affects the quality of the generated SQL. However, standard generative guidelines are often too generic, especially when dealing with translation tasks of varying difficulties. Therefore, we employ difficulty-aware guidelines to guide the LLM in generating SQL queries.

For different difficulty levels d, we include the generative guideline I_d in the prompt, requiring the LLMs to generate a specific type of SQL query. We have designed different generative guidelines based on the difficulty classification of the input NL questions:

- **Simple Category:** No need to specify structural information explicitly; in most cases of this difficulty, LLMs can generate SQL queries based on the problem without guidelines on the SQL category.
- **Multi-table Category:** Instruct the model that the SQL query needs to use multiple tables.
- **Nested Category:** Instruct the model that the SQL query requires nested queries.
- **Complex Category:** A combination of the above two circumstances. Instruct the model to generate SQL queries involving multiple tables and nested queries.

It should be noted that these generative guidelines are added at the beginning of the prompt and do not affect the overall structure of the prompt.

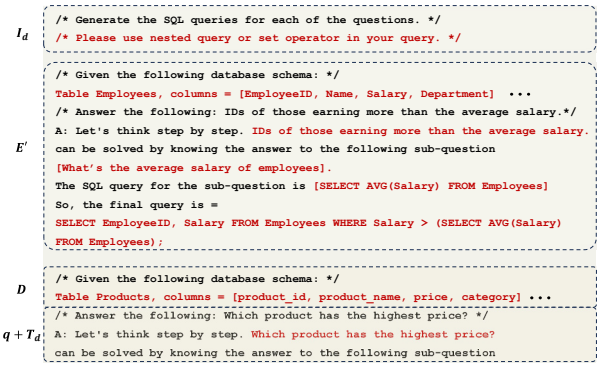


Fig. 3: An example of prompt construction for nested categories. The arrangement from top to bottom includes the generative guideline I_d, CoT-enhanced examples E', database schema D, the NL question q, and the CoT template T_d. In I_d, the red text specifies the guideline, indicating the SQL type that varies depending on the SQL's difficulty. In E', the black text represents the difficulty-specific CoT template, while the red text denotes few-shot example information e_i, such as q_i, db_i, and ss_i. In D, the red text highlights the table schema for the NL question q. In $q + T_d$, the red text corresponds to q, while T_d is left blank for the LLM to complete during reasoning. For demonstration purposes, we have included only one few-shot example in the figure, and the schema of only one table is displayed.

CoT-Enhanced Example Generation. To boost the efficiency and accuracy of information extraction, all examples are methodically organized into a structured knowledge base, B. Within this base, data is stored as tuples formatted as $e_i = \langle q_i, s_i, d_i, db_i, do_i, t_i, sq_i, ss_i \rangle$ (for $i = 1, 2, \ldots$), where each element satisfies the condition $e_i \in e$. Specifically, q_i denotes the question of example e_i, s_i indicates the corresponding SQL query, d_i categorizes the difficulty level, db_i reveals the associated database information, do_i includes domain-specific elements such as vocabulary used within the question, t_i specifies the table utilized in the SQL query s_i, sq_i refers to any sub-questions that break down or elaborate on parts of q_i, and ss_i represents the sub-SQL queries derived from or decomposing s_i. This structured collection of information forms a comprehensive dataset for each example, serving as foundational knowledge to facilitate the Text-to-SQL translation process.

In resolving Text-to-SQL translation, simple reasoning often does not allow LLMs to perform in-depth analysis of the

Authorized licensed use limited to: ULAKBIM UASL - SABANCI UNIVERSITY. Downloaded on October 12,2026 at 15:59:47 UTC from IEEE Xplore. Restrictions apply.

SQL structure. The successful application of Chain of Thought reasoning in LLMs [16] indicates that using CoT can more effectively guide the LLMs in reasoning according to the logical structures of SQL. However, a single CoT template often fails to adapt to different Text-to-SQL translations. For different SQL difficulty levels, designing difficulty-adaptive CoT templates to guide the LLMs can significantly enhance the Text-to-SQL translation performance.

We propose a difficulty-adaptive CoT construction strategy that combines few-shot examples $e = \{e_1, e_2, \ldots, e_n \mid e_i \in B\}$ obtained through the example retrieval module with CoT templates T_d. During the retrieval process for few-shot examples, retrieving examples from B allows us to obtain high-quality examples, enabling our method to achieve sufficiently excellent results. This is due to the effectiveness of the ranking model. Furthermore, it is feasible to leverage the difficulty level for pre-filtering the knowledge base B to obtain a refined subset B', followed by performing retrieval within B'. This approach can yield marginal accuracy improvements. For the predefined difficulty levels d, we define corresponding CoT templates T_d, each tailored with specific reasoning structures according to the difficulty level. After retrieving few-shot examples E using the difficulty, we populate the corresponding difficulty's CoT template with few-shot examples to form enhanced CoT examples E'. This process can be expressed as:

$$E' = \mathcal{C}(e, T_d) \tag{4}$$

$\mathcal{C}$ represents the process of combining retrieved few-shot examples E with the CoT template T_d to construct CoT-enhanced examples. In this setup, the template T_d defines the framework of the difficulty-aware reasoning process. Instance information in the framework, such as question text q, ground-truth SQL query s_t, and database information D, needs to be dynamically populated during the construction. The information used in the CoT-enhanced examples is a subset of all the information in e_i. The specific design of T_d is as follows:

- **Simple Category:** Since the task is relatively simple, this difficulty level does not require a CoT derivation process. Therefore, the CoT template for this difficulty contains only the question and SQL from the few-shot example, to construct the simplest Text-to-SQL translation process. For each example e_i, the information needed is $\langle q_i, s_i, db_i \rangle$
- **Multi-table Category:** This CoT template guides the LLM to recognize domain information in the question and deduce the tables and columns required in the SQL query, finally building the SQL based on these schemas. The example information needed is $\langle q_i, s_i, db_i, do_i, t_i \rangle$
- **Nested Category:** The CoT template first guides the LLM to generate sub-questions sq_i contained in question q_i, and from these sub-questions deduce corresponding sub SQL queries ss_i. These queries are the sub-queries of the final SQL query, which are then integrated into the SQL query to generate the SQL statement. For each example e_i, the information needed is $\langle q_i, s_i, db_i, sq_i, ss_i \rangle$
- **Complex Category:** Although this difficulty includes both multiple tables and sub-queries, sub-queries play a more critical role in the construction of SQL. Therefore, the CoT template for this difficulty level is the same as that for the nested level, with a focus on generating sub-queries.

These structures can be integrated into the Prompt as auxiliary information for the LLM, enhancing the model's performance in handling Text-to-SQL translation. Based on the difficulty d determined by the difficulty classification model DCM, we obtain the generative guidelines I_d and CoT template T_d corresponding to the difficulty. We then incorporate few-shot examples E into T_d to generate CoT-enhanced examples E'. Finally, we construct the prompt using the elements above. The inputs also include some other components: NL question q, database information D such as table schema and foreign keys, and a CoT template T_d to guide the LLM in initiating the reasoning process. The goal is to obtain the target SQL Y. This process can be expressed as:

$$\hat{Y} = \mathcal{LLM}(I_d, E', D, q, T_d) \tag{5}$$

In this function, $\mathcal{LLM}$ represents the invocation of the LLM. As shown in Fig. 3, in the final prompt, the generative guideline I_d comes first, followed by E', then D, q, and finally the CoT Template T_d corresponding to the difficulty.

C. Few-shot Example Retrieval for RAG

This section aims to retrieve high-quality examples from the knowledge base B, which will be used as few-shot examples to guide LLMs in solving Text-to-SQL translation. We first reorganize the examples in the training dataset of the benchmarks to form the knowledge base B. We then train a ranking model based on the reorganized B. For an NL question q, we use the ranking model to predict the quality scores between it and the examples in B, and we retrieve the examples with the top quality scores as the few-shot examples. Fig. 4 demonstrates the overall training process of the model.

In our retrieval strategy, E is a set of examples retrieved from B, which are similar to the ground-truth SQL query s_t of the NL question input. These examples in E are obtained dynamically through our retrieval strategy. This process can be depicted as $E = \text{Retrieve}(q, R, B) = \{e_1, e_2, \ldots, e_n | e_i \in B\}$, where e_i denotes a specific example, R denotes the retrieval strategy, and Retrieve denotes the process of retrieving using the strategy R. The overall retrieval goal is to retrieve examples that are more relevant to the NL question q and its corresponding ground-truth SQL s_t. This means that when these examples are used in a prompt, they can effectively facilitate the generation of the final Text-to-SQL translation. We define such examples as few-shot examples with high quality.

Example Retrieval via Ranking Model. A common phenomenon has been observed [31] that the more similar the SQL query s_i of a few-shot example e_i is to the ground-truth SQL s_t of a target problem t, the more pronounced the performance improvement becomes for Text-to-SQL translation. Here the

Authorized licensed use limited to: ULAKBIM UASL - SABANCI UNIVERSITY. Downloaded on October 12,2026 at 15:59:47 UTC from IEEE Xplore. Restrictions apply.

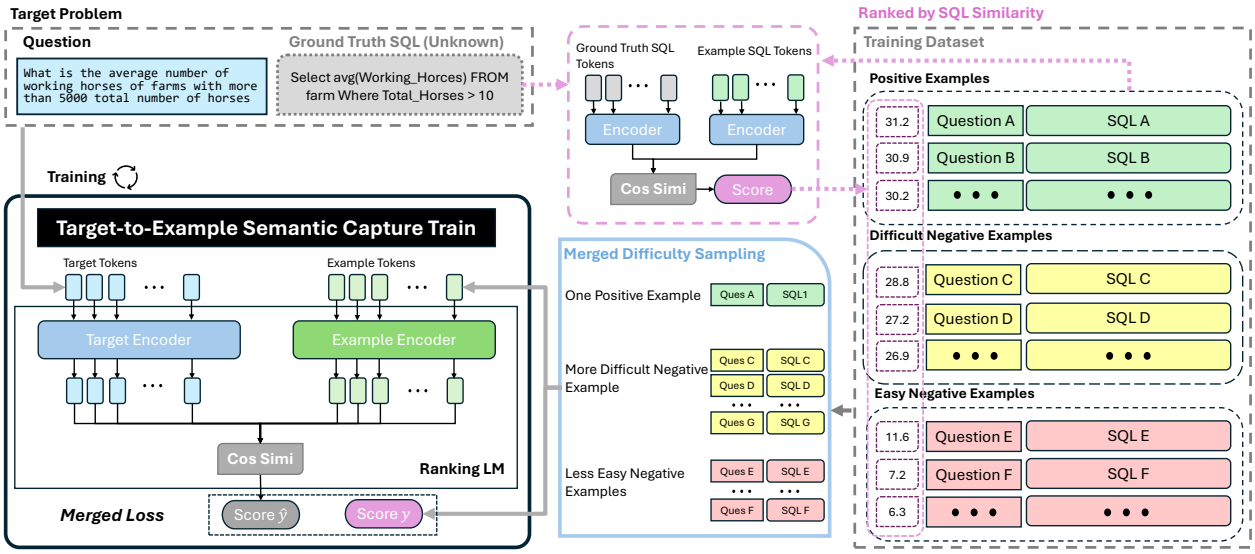


Fig. 4: Overall of the proposed ranking model training framework for approximately high-quality few-shot demonstrated example retrieval. It can optionally assemble NL questions and SQL queries as token inputs of the model. We show the evaluation results of different input text assembling strategies in Fig. 6

target problem t refers to the current problem we need to resolve, which includes a known NL question q and the ground-truth SQL query s_t that we cannot obtain. This suggests that the semantic and structural similarity between the few-shot example e_i and the ground-truth SQL query s_t directly impacts the LLM's performance in Text-to-SQL translation. Therefore, our goal is to improve the relevance of few-shot examples.

Using the ground-truth SQL s_t of t for semantic similarity retrieval allows for obtaining relatively high-quality e_i. However, s_t is unavailable, making it difficult to precisely retrieve high-quality e_i from B. Moreover, existing retrieval strategies rely on LM encoders pre-trained on large general corpora, which lack the specific knowledge required for Text-to-SQL translation.

We propose a dedicated ranking-model-based strategy to improve the quality of examples retrieved, thus optimizing the contribution of few-shot examples to Text-to-SQL translation. Given a target problem t, we use the ranking model to predict the approximate quality scores for all candidate examples in B, then sort them by their scores, and select a small subset of examples with the highest scores as few-shot examples E. Our ranking model mainly consists of two BERT-based encoder parts: **T_Encoder**, which is used to extract semantic features from the target problem t, and **E_Encoder**, which is used to extract semantic features of example e_i from B.

Due to the significant differences in semantics and structure between NL questions and SQL queries, relying solely on NL questions for example retrieval is insufficient. Since we cannot obtain the ground-truth SQL query s_t, we pre-generate SQL queries s_p using the pre-trained language model defined in Section III-B, which, to some extent, represents the information of the ground-truth SQL query. Thus, the target problem t now consists of three parts: a known NL question q, an unavailable ground-truth SQL query s_t, and a pre-generated SQL query s_p. The database schema plays a crucial role in SQL generation and can significantly impact the perceived similarity of SQL-related questions. For the same natural language query, different schemas may lead to varying levels of generation difficulty. However, quantifying how schema structures affect text-to-SQL similarity remains challenging, as does modeling the relationship between schema similarity and ultimate query construction complexity. Due to these difficulties, we currently exclude schema considerations from our retrieval process. That said, we acknowledge that effective schema representation could potentially enhance retrieval performance in future implementations.

Overall, the ranking model takes two texts as input: the first text consists of the concatenation of the NL question q and its pre-generated SQL query s_p; the second part is the SQL query s_i of the example e_i in the knowledge base B. Both parts are fed into encoders, where q and s_p are fed into the **T_Encoder**, and s_i is fed into the **E_Encoder**, producing features as a type of vector sequence, which is denoted as F_t and F_e.

$$F_t = \text{T_Encoder}(\langle q, s_p \rangle), \quad F_e = \text{E_Encoder}(s_i)$$
$$\hat{y} = \sum_{f_t \in F_t} \max_{f_e \in F_e} \text{Cos}(f_t, f_e) \tag{6}$$

The vectors in both sequences have the same dimensions, with each vector in F_t denoted as f_t, and each vector in F_e as f_e. After processing by the encoders, we calculate the maximum pooling of the cosine similarity between each vector f_t in F_t and each vector f_e in F_e to determine the final predicted score. The calculation process for the predicted score $\hat{y}$ is formulated in Equation 6.

Authorized licensed use limited to: ULAKBIM UASL - SABANCI UNIVERSITY. Downloaded on October 12,2026 at 15:59:47 UTC from IEEE Xplore. Restrictions apply.

Since our ranking model has a more fine-grained representation of NL (natural language) questions and SQL queries, it exhibits a stronger ability to capture related semantic-specific knowledge, which enables it to retrieve few-shot examples with higher quality.

Construction of the Ranking Dataset. To train our ranking model, we establish the dataset for model training using examples from the knowledge base B. Then, we process the training dataset to optimize training performance.

The definition of example quality is the core issue we need to consider. Based on the idea of using semantic similarity between SQL queries as an approximate measurement of quality, for example e_i and target problem t, we define the approximate quality score $\sigma(e_i, t)$ as follows:

$$\sigma(e_i, t) = \text{Cos}(\text{Encoder}(s_i), \text{Encoder}(s_t))) \tag{7}$$

The Encoder denotes the process of text-embedding through a pre-trained language model, which encodes a text token input into a single vector representation containing semantic information. Cos denotes the process of calculating the cosine similarity between vector representations. We then adjust the cosine similarity value to a new range, which is more suitable for model training.

In this way, we define how to evaluate the few-shot quality of e_i for t in approximate terms. A higher score indicates that the SQL query s_i for example e_i is semantically more similar to the ground truth SQL s_t of t. In our definition, an example e_i in the knowledge base B with a higher score is considered to be a higher-quality example.

The objective of the training is to enable the ranking model to output a higher score for higher-quality examples and a lower score for lower-quality examples given a target problem t. The quality values are approximately measured by the quality score function, Equation 7, which reflects the semantic similarity between SQL queries. In the reorganized training dataset, which is of size N, each e_i is used as a target problem t_i, with the remaining $N-1$ examples serving as the corresponding knowledge base B_i. For each t_i, we then calculate the quality score $\sigma(e_j, t_i)$ for every e_j in B_i, which serves as the score label y_{ij}. Consequently, for each t_i, we generate $N-1$ pairs $\langle t_i, e_j, y_{ij}\rangle$ to train the ranking model. Given the large magnitude of N^2 training pairs constructed in this manner, we randomly sample n examples from t_i's corresponding B_i when training, rather than using them all.

Training with Merged Difficulty Sampling. During the training process, it is necessary to label examples in the knowledge base B as either positive or negative. For a target problem t_i selected from B for training, the top k examples with the highest scores from the knowledge base B_i are designated as positive, denoted by $E_i^{\text{pos}} = \langle e_{i1}^{\text{pos}}, e_{i2}^{\text{pos}}, \ldots, e_{ik}^{\text{pos}}\rangle$. All other examples are labeled as negative. However, a challenge arises during this differentiation: distinguishing between E_i^{pos} and negative examples with high scores (which are semantically closer to the positives) proves more difficult than differentiating them from negative examples with low scores. Resolving this issue is a crucial aspect of our training process.

Therefore, we propose a strategy of merged-difficulty training: rather than dividing the knowledge base B_i into positive and negative types, the examples in B_i are divided into three categories: positive examples E_i^{pos}, difficult-to-distinguish negative examples E_i^{diff}, and easy negative examples E_i^{easy}. During the sampling phase of training, these three categories are assigned different sampling weights. Specifically, for a target problem t_i, a positive example e_{ij}^{pos} is paired with a greater number of E_i^{diff} (which is assigned a higher sampling weight) and a fewer number of E_i^{easy} (which is assigned a lower sampling weight), forming a randomly sampled pair such as $\langle t_i, e_{ij}^{\text{pos}}, E_i^{\text{diff}}, E_i^{\text{easy}}\rangle$. This design increases the proportion of difficult examples in training, enabling the model to better learn to distinguish high-quality examples.

First, referring to Equation 6, for a target t_i and one example e_{ij} from its corresponding merged sampled pair, we denote the score between t_i and e_{ij} predicted by model R as $\hat{y}_{ij} = \mathcal{P}(\langle q_i, s_{pi}\rangle, s_j; \text{R})$. Here, s_j represents the SQL query of example e_{ij}, $\mathcal{P}$ denotes the process of score prediction, and y_{ij} denotes the label score between t_i and e_{ij} as mentioned earlier in the construction of training datasets.

In designing the loss function, we use two types of loss for E_i^{diff} and E_i^{easy}, respectively. For each e_{ij} in E_i^{easy}, a one-hot label of 0 is assigned to replace its original score label y_{ij}, while a one-hot label of 1 is assigned to e_{ij}^{pos}. We denote these new 0-1 one-hot labels as $y_{\text{ij}}^{\text{easy}}$. Then, the CrossEntropy loss is computed between the new $y_{\text{ij}}^{\text{easy}}$ and the model's predicted scores after softmax. However, for E_i^{diff}, assigning a label of 0 would lead to contradictory training guidance due to their semantic closeness to positive examples. Therefore, we directly calculate the loss between the score labels y_{ij} and the model predicted scores $\hat{y}_{ij}$ via KL Divergence loss. Assuming the number of examples in the pair is $|\langle e_j^{\text{pos}}, E_i^{\text{diff}}, E_i^{\text{easy}}\rangle| = n$, the design of the loss function for a pair is as follows:

$$\begin{aligned} L_{\text{easy}} &= -\frac{1}{n}\sum_{j=1}^{n} y_{ij} \cdot \log(\hat{y}_{ij}) \\ L_{\text{diff}} &= \frac{1}{n}\sum_{j=1}^{n} y_{ij} \cdot \log\frac{y_{ij}}{\hat{y}_{ij}} \\ L_{\text{rank}} &= \lambda \cdot L_{\text{easy}} \ + \ L_{\text{diff}} \end{aligned} \tag{8}$$

for a pair $\langle t_i, e_{ij}^{\text{pos}}, E_i^{\text{diff}}, E_i^{\text{easy}}\rangle$ of t_i, we calculate L_{easy} and L_{diff} respectively on E_i^{easy} and E_i^{diff}, and then use the weight λ to adjust the value of L_{easy}, making the values of the two losses close to each other. Finally, we use the sum of the two as the overall loss.

This design explicitly allows the model to penalize scores for easy negative examples and utilize a fine-grained score-fitting loss for challenging difficult negative examples. This strategy helps avoid training instability caused by rigid labels and enables the model to better capture subtle differences among difficult examples, thereby enhancing overall generalization capabilities.

Finally, we can demonstrate our target for training the ranking model. Given the ranking model R and a set of pairs

Authorized licensed use limited to: ULAKBIM UASL - SABANCI UNIVERSITY. Downloaded on October 12,2026 at 15:59:47 UTC from IEEE Xplore. Restrictions apply.

as training data $\mathcal{T} = \{\langle t_i, e_{ij}^{\text{pos}}, E_i^{\text{diff}}, E_i^{\text{easy}} \rangle\}$, the objective of the training is to minimize the following overall loss.

$$\min_{R} \sum_{\langle t_i, e_j^{\text{pos}}, E_i^{\text{diff}}, E_i^{\text{easy}} \rangle \in \mathcal{T}}^{|\mathcal{T}|} L_{\text{rank}}(t_i, e_j^{\text{pos}}, E_i^{\text{diff}}, E_i^{\text{easy}}) \quad (9)$$

where L_{rank} is the merged loss function to measure the difference between the model predicted score $\hat{y}_{ij}$ of $\langle t_i,\ e_j \rangle$ and their label score y_{ij}.

IV. Experiment

A. Experimental Setup

Benchmarks. We evaluate our method on five popular Text-to-SQL benchmarks: Spider [32], Spider-DK [33], Spider-SYN [34], Spider-Realistic [35] and BIRD [30].

- **Spider** contains 10,181 questions and 5,693 unique complex SQL queries, involving 200 databases across multiple tables covering 138 different domains.
- **BIRD** includes 12,751 question-SQL pairs and 95 large databases across 37 domains, focusing on large-scale noisy data and external knowledge reasoning.
- **Spider-DK** focuses on domain-specific knowledge in SQL generation, introducing stringent domain-specific requirements for Text-to-SQL translation strategies.
- **Spider-Realistic** addresses text-to-table alignment challenges by omitting explicit column mentions, requiring NL terms to be mapped to the relevant database schema.
- **Spider-SYN** tests lexical matching dependence by substituting schema-related terms with synonyms, challenging LLMs' comprehension of NL questions.

Baselines. To evaluate the performance of AID-SQL, we compare it with Text-to-SQL approaches based on PLM or LLM. **RESDSQL** [18] is a competitive PLM-based method that decouples schema linking and skeleton parsing. **C3** [25] is a zero-shot method that utilizes carefully crafted instructions to reduce input length for LLM API calls. **DIN-SQL** [19] employs few-shot learning and incorporates CoT reasoning to enhance performance. **DAIL-SQL** [15] adopts an adaptive demonstration selection strategy by analyzing semantic similarities between natural language queries and SQL statements. **CHESS** [36] is a multi-agent framework designed to support industrial-grade databases while minimizing the number of LLM calls. **PET-SQL** [37] implements a two-stage approach, combining advanced prompt engineering with cross-consistency validation. **PURPLE** [38] leverages innovative context retrieval and schema filtering techniques to ensure robust performance in complex scenarios. **RSL-SQL** [39] integrates bidirectional schema linking, contextual information augmentation, binary selection strategy, and multi-turn self-correction mechanisms. **SuperSQL** [40] optimizes the combination of modules through an automated architecture search algorithm. **MCS-SQL** [14] leverages multiple prompts and multiple-choice selection to enhance the accuracy and robustness of query generation. **Distillery** [41] makes full use of LLM's context window, reduces column filtering, and employs enhancement, selection, and correction techniques.

Evaluation Metrics. We employ two evaluation metrics to assess the performance of methods: *Exact-Set Match (EM)* accuracy and *Execution (EX)* accuracy. The detailed description of the metrics is as follows:

- EM accuracy evaluates SQL predictions by comparing each clause of the predicted query with the corresponding clause in the reference query. A predicted SQL is deemed correct only if all its components exactly match the ground-truth SQL.
- EX accuracy compares the output of the predicted SQL query with the output of the ground-truth SQL query executed on a database instance.

Implementation Details. For all LLMs, we adopt a maximum context length of 4096. A fine-tuned T5 model is employed to pre-generate SQL queries. For difficulty classification, we utilize a BERT model [8] with a fully connected layer as the classification head. The model is fine-tuned using questions and SQL texts from public benchmark datasets, with preprocessed SQL category types as classification labels. During training, we configured a linear decay scheduler starting from 2e-5 for the learning rate and set the weight decay rate to 0.01. The model is trained for 30 epochs on both the Spider training and BIRD training datasets.

The ranking model is trained using the architecture of ColBERT model [42]. Our model is trained for 30 epochs on training datasets, with a learning rate initialized at 1e-5 and decayed linearly. Each training batch consists of 32 examples. For a given example, the similarity between it and the other examples in the knowledge base is calculated and ranked. The example with the highest similarity is selected as the positive sample, while those with lower similarity are labeled as negative samples. During the experimental phase, We employ the schema filtering mechanism used in RESDSQL [18]. We also incorporate the correction mechanism into our framework, where SQL queries detected as incorrect by LLM will be regenerated and revised.

B. Overall Performance

TABLE I: Comparison of EX scores on Spider benchmark.

Methods	Dev	Test
RESDSQL-3B [18]	84.1	79.9
C3 + ChatGPT [25]	81.8	82.3
DIN-SQL + GPT-4 [19]	81.8	85.3
DAIL-SQL + GPT-4 [15]	83.6	86.6
SuperSQL [40]	87.0	-
CHESS$_{IR+SS+CG}$ [36]	-	87.2
PET-SQL [37]	-	87.6
PURPLE + GPT-4 [38]	-	87.8
RSL-SQL + GPT-4o [39]	-	87.9
MCS-SQL + GPT-4 [14]	**89.5**	**89.6**
AID-SQL + ChatGPT	82.4	84.5
AID-SQL + GPT-4	<u>85.3</u>	88.3
AID-SQL + GPT-4o	85.1	<u>88.5</u>

In Table I, we present the EX accuracy of our method alongside various previous methods on the Spider benchmark.

Authorized licensed use limited to: ULAKBIM UASL - SABANCI UNIVERSITY. Downloaded on October 12,2026 at 15:59:47 UTC from IEEE Xplore. Restrictions apply.

Previous researches, such as PURPLE, DAIL-SQL, MCS-SQL, and RSL-SQL, have demonstrated outstanding performance. Our findings demonstrate that our method achieves competitive performance regarding EX accuracy on the Spider benchmark. Specifically, when employing GPT-4o, our approach obtains an EX accuracy of 88.5% on the Spider test dataset. Although MCS-SQL achieves higher accuracy on the Spider test dataset, it relies on a self-consistency mechanism, which incurs higher computational overhead due to increased token consumption. Notably, our method (which utilizes ChatGPT) delivers performance comparable to DIN-SQL (which utilizes GPT-4), while being more cost-effective.

We further evaluate our framework on the BIRD benchmark, as shown in Table II. To address the complexity of the BIRD benchmark, we incorporate a self-consistency mechanism and use the template tailored for the simple level as the prompt. The results demonstrate that AID-SQL achieves competitive performance using GPT-4o. This performance improves to 70.08% EX accuracy when integrated with the M-schema module from XiYan-SQL [43]. We further integrate AID-SQL with the RSL-SQL baseline, achieving a 2.29% improvement in EX accuracy through optimized prompt construction. This improvement highlights the effectiveness and adaptability of AID-SQL. Experimental results demonstrate that our RAG module effectively serves as an auxiliary component for retrieving higher-quality examples, thereby enhancing the efficacy of the few-shot learning paradigm.

TABLE II: Comparison of EX scores on BIRD dev dataset.

Methods	Dev
RSL-SQL + DeepSeek [39]	63.56
+AID-SQL	65.85(+2.29)
GPT-4	46.35
DIN-SQL + GPT-4 [19]	50.72
DAIL-SQL + GPT-4 [33]	54.76
SuperSQL [40]	58.5
PURPLE + GPT-4o [38]	62.97
MCS-SQL + GPT-4 [14]	63.36
Distillery + GPT-4o [41]	67.21
CHESS$_{IR+CG+UT}$ [36]	68.31
AID-SQL + GPT-4o	64.32
AID-SQL + M-schema [43] + GPT-4o	**70.08**

As shown in Fig. 5, our method achieves the highest EX accuracy in all three benchmarks, with scores of 77.5%, 78.0%, and 83.1% on Spider-DK, Spider-SYN, and Spider-Realistic, respectively. It demonstrates the robustness of our method across various benchmark tests. The figure also shows that our method attains admirable EM scores, showcasing the remarkable ability of our method.

C. Performance with Various LLMs

To assess the performance of our method across different LLMs, we conduct experiments with five different LLMs: ChatGPT, GPT-4, Llama3-70B, DeepSeek-v3, and Qwen2.5 utilizing the Spider dev dataset for evaluation. As shown in Table III, all models exhibit enhanced performance when transitioning from zero-shot learning to few-shot learning, demonstrating the significance of few-shot examples in boosting model accuracy. Our method substantially outperforms both zero-shot and few-shot strategies across all three models. Notably, our method surpasses the few-shot performance of ChatGPT by 6.7% in accuracy, and surpasses the few-shot performance of GPT-4 by 8.5%, demonstrating its effectiveness in improving Text-to-SQL translation. The degree of improvement varies based on the LLMs' inherent capabilities.

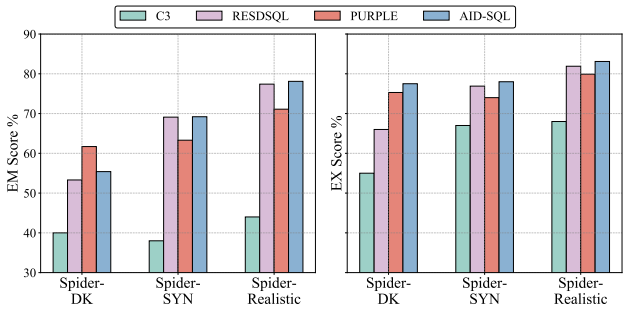


Fig. 5: Comparison of EM/EX scores on Spider-DK, Spider-SYN and Spider-Realistic. Among these methods, RESDSQL used a fine-tuned T5 model, whereas the others relied on ChatGPT.

TABLE III: EX comparison across different LLMs on Spider dev dataset.

Methods	ChatGPT	GPT-4	Llama3	DeepSeek-v3	Qwen2.5
Zero-shot	70.1	72.9	71.3	73.8	73.4
Few-shot	75.7	76.8	74.4	77.2	76.6
AID-SQL	**82.4**	**85.3**	**80.5**	**85.1**	**84.7**

D. Comparison of Difficulty Classification Strategies

TABLE IV: Comparison of difficulty classification performance of different strategies on Spider dev dataset.

Methods	Token Consumption	Accuracy
LLM Prompt	2,286	76.8%
GPT-3.5 Pre-generation	927	80.2%
GPT-4 Pre-generation	927	81.4%
PLM Pre-generation	0	77.6%
Our Strategy	**0**	**95.8%**

Table IV lists the average classification accuracy and the LLM token consumption per question for various classification strategies on the Spider dev dataset. The LLM Prompt method (used in DIN-SQL [19]) involves designing a prompt that enables the LLM to deduce the category of the question. The pre-generation method refers to pre-generating SQL queries in advance, and then determining the category of the NL question through semantic and structural analysis of the SQL. Our method achieves a remarkable classification accuracy of 95.8% without consuming any LLM tokens, significantly surpassing

Authorized licensed use limited to: ULAKBIM UASL - SABANCI UNIVERSITY. Downloaded on October 12,2026 at 15:59:47 UTC from IEEE Xplore. Restrictions apply.

the 81.4% accuracy of the GPT-4 pre-generation method, marking a substantial improvement in classification accuracy. Pre-generation methods rely on more capable models like GPT-4 to pre-generate SQL to enhance classification accuracy. However, this approach is costly due to high token consumption. Although using a small-scale PLM for pre-generation can eliminate the need for LLM tokens, it compromises accuracy. DIN-SQL's classification strategy consumes an average of 2286 tokens and only reaches an accuracy of 76.8%, which is neither economical nor efficient. In contrast, our difficulty classification strategy successfully balances token consumption and classification accuracy.

TABLE V: Ablation Study of AID-SQL.

Strategy	Simple	Multi-table	Nested	Complex	All
AID-SQL(ChatGPT)	92.4	73.4	69.9	71.1	82.4
all easy prompt	92.4	69.5	65.1	68.4	81.1
all hard prompt	90.7	67.9	67.1	71.1	80.1
-guidelines	92.4	68.4	66.8	70.2	81.0
-example difficulty filter	92.4	72.7	69.2	70.8	82.1
-RAG	89.0	67.4	61.4	69.7	78.4

E. Ablation Study of AID-SQL

In the ablation study, we assess our method both with and without each of the modules. As illustrated in Table V, omitting either module results in a decline in performance, particularly in overall EX accuracy.

In the absence of the classification module, the same CoT template is applied to all NL questions regardless of their difficulty. We use either simple or complex prompts for all NL questions. The experimental findings indicate that removing the classification module adversely impacts accuracy, regardless of the prompt type. A simple prompt refers to a scenario without CoT reasoning or generative guidelines, using basic prompts for questions of varying difficulty levels. In contrast, a complex prompt incorporates generative guidelines and adopts a least-to-most CoT template to systematically guide SQL generation. Compared to the comprehensive framework, omitting the difficulty classification module results in lower accuracy. Notably, a complex prompt with CoT is more effective for nested and set queries, while a simple prompt is better suited for simple queries. Removing the generative guidelines(at row: -guidelines) leads to a decrease in EX accuracy to 81.0%. Since simple difficulty questions do not use guidelines, their accuracy remains unchanged. However, the other three difficulty levels are somewhat impacted.

The removal of the example difficulty filter (at row: -example difficulty filter) indicates that no filtering is applied to the knowledge base before retrieving few-shot examples. The results show that after removing the difficulty filter, the accuracy of our method decreases by 0.3%, but it still reaches 82.1%, which is still very competitive. This indicates that the difficulty filter contributes to our few-shot retrieval, but even without it, the ranking model can still demonstrate excellent performance. Removing the RAG module (at row: -RAG) results in a substantial 4% decrease in accuracy, demonstrating the critical role of our dedicated ranking model in the retrieval process.

TABLE VI: The comparison of different retrieval strategies. Recall refers to Top5 Recall, and accuracy is measured using EX accuracy. Tested on the Spider dev dataset.

Selection	Similarity	Recall	ChatGPT	GPT-4	Llama3-70B
Random	0.3149	0.0019	74.2	79.5	73.9
Question	0.6161	0.6886	74.9	79.9	74.8
DAIL-SQL [15]	0.6415	0.7573	75.2	82.4	75.4
Ground-truth SQL	**0.7382**	–	76.8	83.4	77.1
Our Strategy	0.6838	**2.1508**	**77.4**	**83.7**	**77.6**

F. Comparison of Different Retrieval Strategies

To evaluate the quality of few-shot examples retrieved through different strategies, we uses three metrics: cosine similarity, top 5 recall count, and EX accuracy. Cosine similarity measures the average similarity between the SQL of retrieved examples and the ground-truth SQL after text embedding. Recall count indicates how many retrieved examples match those retrieved using the ground-truth SQL.

Furthermore, we employ the ground-truth SQL query to obtain examples as a reference, which represents the upper performance bound for similarity-based retrieval strategies. All tests are conducted on the Spider dev dataset using the prompt structure of the simple category without guidelines and CoT template, utilizing a 5-shot approach. Table VI compares different example retrieval strategies, demonstrating that our proposed strategy generally outperforms others, including the ground-truth SQL similarity.

Equipped with GPT-4, our retrieval strategy achieves an 83.7% EX accuracy, surpassing the ground-truth SQL similarity by 0.3%. Although ground-truth SQL similarity is theoretically the upper limit, our results suggest that cosine similarity alone can not fully capture the quality of examples. This is due to a gap between similarity metrics and the actual helpfulness of retrieved examples for Text-to-SQL translation, stemming from semantic and structural differences between SQL queries and questions. Our ranking model addresses this gap by better understanding these differences, thereby aligning example selection more closely with ground-truth SQL. From Table VI, it is evident that while our strategy achieves the highest accuracy, it does not yield the greatest similarity. However, higher recall counts correlate with greater EX accuracy, indicating that recall better represents example quality than cosine similarity.

G. Comparison of Different Ranking Model Training Methods

We evaluate ranking model training strategies in two aspects: negative example sampling and input text assembly. The model is trained on Spider train datasets and evaluated on the Spider dev dataset using two metrics: cosine similarity and top-5 recall.

As shown in Fig. 6, we compare three sampling strategies: (1) random sampling, (2) merged difficulty with 0-1 discrete

Authorized licensed use limited to: ULAKBIM UASL - SABANCI UNIVERSITY. Downloaded on October 12,2026 at 15:59:47 UTC from IEEE Xplore. Restrictions apply.

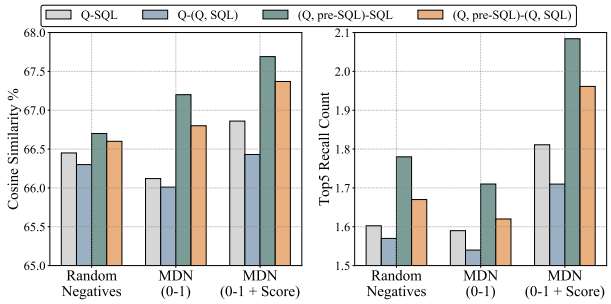


Fig. 6: Evaluation of Different Ranking Model Training Strategies. Here, Q denotes Question, MD (0-1) represents merged difficulty with 0-1 discrete labels, and MD (0-1 + Score) represents merged difficulty with both 0-1 discrete and score-fitting labels.

labels, and (3) merged difficulty with both 0-1 discrete and score-fitting labels. Random Negative Sampling selects negative examples uniformly from all available negatives. The merged difficulty strategy splits negative examples into easy and difficult categories based on their semantic similarity to the positive example, sampling a larger proportion of difficult negatives. The first variant assigns discrete 1-labels to positive examples and 0-labels to negative examples for loss calculation. The second variant additionally uses soft similarity scores as labels, providing more granular supervision. We also compare four strategies for assembling Query and Example texts: Question-SQL, Question-(Question, SQL), (Question, pre-SQL)-SQL, and (Question, pre-SQL)-(Question, SQL). These strategies specify how NL questions are used for retrieval, with the text before the '-' indicating the query and the text after the '-' representing the retrieved example.

Results show that using merged difficulty sampling improves performance by including semantically closer negative examples. Compared to random sampling, cosine similarity increases from 66.5% to 66.9%, and *Top5* accuracy improves by 9%. However, using 0-1 discrete labels with merged difficulty leads to a performance drop. Based on these strategies, we train ranking models respectively. Integrating pre-generated SQL into Query texts enhances retrieval performance, improving cosine similarity by 1%. Conversely, adding question texts to retrieved Examples decreases all three metrics, regardless of pre-SQL usage.

H. Example Analyses

By analyzing the issues encountered during Text-to-SQL translation, we find that LLMs tend to generate relatively simple SQL queries and lack depth in exploring the structural intricacies of SQL. This limitation can lead to structural deviations in the generated SQL. As illustrated in Figure 7, we present two examples to demonstrate the effectiveness of our approach in addressing this issue.

In the SQL query for Question 1, the LLM fails to capture the emphasis on the word "product" in the question and instead

Question 1:
Which product has the highest sales?

Answer of Standard Prompt:
SQL: SELECT MAX(sales) FROM products;

Answer of Prompt with CoT:
A: Let's think step by step. From the key entities in the question: ['product', 'sales'], we need to use the following schemas: ['product_id', 'sales', 'sales_record']. We create the SQL based on schemas.
SQL:SELECT product_id FROM sales_record ORDER BY sales DESC LIMIT 1;

Question 2:
Get the names of customers who have more than five purchases.

Answer of Standard Prompt:
SQL: SELECT customer_name FROM orders WHERE purchase_count > 5;

Answer of Prompt with CoT:
A: Let's think step by step. The question can be solved by knowing the answer to the sub-question [Number of purchases per customer].
The SQL query for the sub-question is [SELECT customer_id, COUNT(*) as purchase_count FROM orders GROUP BY customer_id HAVING COUNT(*) > 5;]. So, the query to the question is as follow
SQL: SELECT customer_name FROM customers WHERE customer_id IN (SELECT customer_id FROM orders GROUP BY customer_id HAVING COUNT(*) > 5);

Fig. 7: This section provides examples where the LLM initially generated incorrect solutions, but our CoT approach successfully resolved them. In question 1, the red highlighted parts indicate the tables and columns involved, with a focus on relationships between tables and columns. In question 2, the red-highlighted part represents a subquery, emphasizing the nested structure that the LLM needs to focus on.

focuses solely on the word "sales". By using the difficulty-aware CoT, the LLM can deduce the required patterns from the keywords "product" and "sale". For Question 2, the LLM incorrectly generates a query that directly references the "purchase_count" field, even though this field needs to be calculated through an aggregation. After introducing the difficulty-aware CoT, the LLM first addresses the sub-question and then combines the results with the "customers" table to form a correct nested query.

V. Conclusion

This paper proposes AID-SQL, an adaptive in-context learning method that leverages difficulty-adaptive instructions and retrieval-augmented generation. Our approach introduces a framework that classifies SQL queries based on multiple dimensions and designs tailored instructions to guide the LLM. Additionally, we enhance the few-shot example retrieval module by training a ranking model. Experimental results demonstrate competitive performance across existing benchmarks, validating the effectiveness of our approach.

Acknowledgment

The authors thank the reviewers for the valuable feedback. This work was supported by National Natural Science Foundation of China (62372179, 62406112).

Authorized licensed use limited to: ULAKBIM UASL - SABANCI UNIVERSITY. Downloaded on October 12,2026 at 15:59:47 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] M. Y. Carriere-Swallow and M. V. Haksar, *The economics and implications of data: an integrated perspective*. International Monetary Fund, 2019.

[2] F. Li and H. V. Jagadish, "Constructing an interactive natural language interface for relational databases," *Proceedings of the VLDB Endowment*, vol. 8, no. 1, pp. 73–84, 2014.

[3] T. Mahmud, K. A. Hasan, M. Ahmed, and T. H. C. Chak, "A rule based approach for nlp based query processing," in *2015 2nd international conference on electrical information and communication technologies (EICT)*. IEEE, 2015, pp. 78–82.

[4] J. M. Zelle and R. J. Mooney, "Learning to parse database queries using inductive logic programming," in *Proceedings of the national conference on artificial intelligence*, 1996, pp. 1050–1055.

[5] S. Hochreiter, "Long short-term memory," *Neural Computation MIT-Press*, 1997.

[6] V. Zhong, C. Xiong, and R. Socher, "Seq2sql: Generating structured queries from natural language using reinforcement learning," *arXiv preprint arXiv:1709.00103*, 2017.

[7] J. Qi, J. Tang, Z. He, X. Wan, Y. Cheng, C. Zhou, X. Wang, Q. Zhang, and Z. Lin, "Rasat: Integrating relational structures into pretrained seq2seq model for text-to-sql," *ArXiv*, vol. abs/2205.06983, 2022.

[8] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[9] P. Yin, G. Neubig, W.-t. Yih, and S. Riedel, "Tabert: Pretraining for joint understanding of textual and tabular data," *arXiv preprint arXiv:2005.08314*, 2020.

[10] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of machine learning research*, vol. 21, no. 140, pp. 1–67, 2020.

[11] Z. Hong, Z. Yuan, Q. Zhang, H. Chen, J. Dong, F. Huang, and X. Huang, "Next-generation database interfaces: A survey of llm-based text-to-sql," *arXiv preprint arXiv:2406.08426*, 2024.

[12] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Nov. 2021, pp. 9895–9901. [Online]. Available: https://aclanthology.org/2021.emnlp-main.779

[13] T. B. Brown, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.

[14] D. Lee, C. Park, J. Kim, and H. Park, "Mcs-sql: Leveraging multiple prompts and multiple-choice selection for text-to-sql generation," *arXiv preprint arXiv:2405.07467*, 2024.

[15] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-sql empowered by large language models: A benchmark evaluation," *arXiv preprint arXiv:2308.15363*, 2023.

[16] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[17] C.-Y. Tai, Z. Chen, T. Zhang, X. Deng, and H. Sun, "Exploring chain-of-thought style prompting for text-to-sql," *arXiv preprint arXiv:2305.14215*, 2023.

[18] H. Li, J. Zhang, C. Li, and H. Chen, "Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, 2023, pp. 13 067–13 075.

[19] M. Pourreza and D. Rafiei, "Din-sql: Decomposed in-context learning of text-to-sql with self-correction," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[20] Z. Chen, L. Chen, Y. Zhao, R. Cao, Z. Xu, S. Zhu, and K. Yu, "Shadowgnn: Graph projection neural network for text-to-sql parser," *arXiv preprint arXiv:2104.04689*, 2021.

[21] J. Guo, Z. Zhan, Y. Gao, Y. Xiao, J.-G. Lou, T. Liu, and D. Zhang, "Towards complex text-to-sql in cross-domain database with intermediate representation," *arXiv preprint arXiv:1905.08205*, 2019.

[22] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers," *arXiv preprint arXiv:1911.04942*, 2019.

[23] Y. Fu, W. Ou, Z. Yu, and Y. Lin, "Miga: a unified multi-task generation framework for conversational text-to-sql," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, 2023, pp. 12 790–12 798.

[24] Z. Gu, J. Fan, N. Tang, L. Cao, B. Jia, S. Madden, and X. Du, "Few-shot text-to-sql translation using structure and content prompt learning," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, pp. 1–28, 2023.

[25] X. Dong, C. Zhang, Y. Ge, Y. Mao, Y. Gao, J. Lin, D. Lou *et al.*, "C3: Zero-shot text-to-sql with chatgpt," *arXiv preprint arXiv:2307.07306*, 2023.

[26] A. Liu, X. Hu, L. Wen, and P. S. Yu, "A comprehensive evaluation of chatgpt's zero-shot text-to-sql capability," *arXiv preprint arXiv:2303.13547*, 2023.

[27] L. Nan, Y. Zhao, W. Zou, N. Ri, J. Tae, E. Zhang, A. Cohan, and D. Radev, "Enhancing few-shot text-to-sql capabilities of large language models: A study on prompt design strategies," *arXiv preprint arXiv:2305.12586*, 2023.

[28] N. Reimers, "Sentence-bert: Sentence embeddings using siamese bert-networks," *arXiv preprint arXiv:1908.10084*, 2019.

[29] Z. Hong, Z. Yuan, H. Chen, Q. Zhang, F. Huang, and X. Huang, "Knowledge-to-sql: Enhancing sql generation with data expert llm," *arXiv preprint arXiv:2402.11517*, 2024.

[30] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo *et al.*, "Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[31] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-sql empowered by large language models: A benchmark evaluation," *CoRR*, vol. abs/2308.15363, 2023.

[32] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," *arXiv preprint arXiv:1809.08887*, 2018.

[33] Y. Gan, X. Chen, and M. Purver, "Exploring underexplored limitations of cross-domain text-to-sql generalization," *arXiv preprint arXiv:2109.05157*, 2021.

[34] Y. Gan, X. Chen, Q. Huang, M. Purver, J. R. Woodward, J. Xie, and P. Huang, "Towards robustness of text-to-sql models against synonym substitution," *arXiv preprint arXiv:2106.01065*, 2021.

[35] X. Deng, A. H. Awadallah, C. Meek, O. Polozov, H. Sun, and M. Richardson, "Structure-grounded pretraining for text-to-sql," *arXiv preprint arXiv:2010.12773*, 2020.

[36] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, "Chess: Contextual harnessing for efficient sql synthesis," *arXiv preprint arXiv:2405.16755*, 2024.

[37] Z. Li, X. Wang, J. Zhao, S. Yang, G. Du, X. Hu, B. Zhang, Y. Ye, Z. Li, R. Zhao *et al.*, "Pet-sql: A prompt-enhanced two-stage text-to-sql framework with cross-consistency," *arXiv preprint arXiv:2403.09732*, 2024.

[38] T. Ren, Y. Fan, Z. He, R. Huang, J. Dai, C. Huang, Y. Jing, K. Zhang, Y. Yang, and X. S. Wang, "Purple: Making a large language model a better sql writer," *arXiv preprint arXiv:2403.20014*, 2024.

[39] Z. Cao, Y. Zheng, Z. Fan, X. Zhang, W. Chen, and X. Bai, "Rsl-sql: Robust schema linking in text-to-sql generation," *arXiv preprint arXiv:2411.00073*, 2024.

[40] B. Li, Y. Luo, C. Chai, G. Li, and N. Tang, "The dawn of natural language to sql: Are we fully ready?" *arXiv preprint arXiv:2406.01265*, 2024.

[41] K. Maamari, F. Abubaker, D. Jaroslawicz, and A. Mhedhbi, "The death of schema linking? text-to-sql in the age of well-reasoned language models," *arXiv preprint arXiv:2408.07702*, 2024.

[42] O. Khattab and M. Zaharia, "Colbert: Efficient and effective passage search via contextualized late interaction over bert," in *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, 2020, pp. 39–48.

[43] Y. Gao, Y. Liu, X. Li, X. Shi, Y. Zhu, Y. Wang, S. Li, W. Li, Y. Hong, Z. Luo *et al.*, "Xiyan-sql: A multi-generator ensemble framework for text-to-sql," *arXiv preprint arXiv:2411.08599*, 2024.

Authorized licensed use limited to: ULAKBIM UASL - SABANCI UNIVERSITY. Downloaded on October 12,2026 at 15:59:47 UTC from IEEE Xplore. Restrictions apply.